

CSE 220 Fourier Exam Preparation Guide

Fourier Series, Continuous Fourier Transform, 2D-CFT, Numerical Verification,
and Exam Coding

Built from the supplied previous-year online questions, the Jan 2026 offline assignment, and the supplied solution code.

What is inside

- Solved walkthroughs of all supplied previous-year questions.
- A focused explanation of this year's Fourier-Series epicycle and 2D-CFT edge-detection assignment.
- Fourteen genuinely new exam-style practice questions derived from the current assignment, with full answers.
- Reusable Python patterns using `np.trapezoid`; no FFT.
- Phase/MSE, indexing, units, interpolation, and separability pitfalls.
- A 30–40 minute exam workflow and a final formula/code checklist.

Contents

1	What the previous questions actually test	1
1.1	Likely question families for this year's exam	1
2	Essential theory you should be able to use immediately	1
2.1	Continuous Fourier Transform convention used in these questions	1
2.2	CFT property sheet	2
2.3	Complex Fourier Series used by the epicycle assignment	2
2.4	2D Continuous Fourier Transform	3
2.5	Spatial frequency intuition	3
3	Numerical Fourier coding patterns to memorize	3
3.1	1D CFT without FFT	3
3.2	Fourier Series coefficient and reconstruction	3
3.3	2D trapezoidal integration axis rule	4
3.4	Phase error: do not subtract raw angles blindly	4
3.5	Interpolation for transformed spectra	4
4	This year's assignment: what every line is doing	4
4.1	Task 1: Fourier epicycles	4
4.2	Task 2: separable 2D-CFT	5
4.3	Why <code>self.u</code> and <code>self.v</code> are not optional	5
4.4	High-pass filter and inverse	5
5	Solved previous-year questions	5
5.1	PY1 - July 2025 A1+A2: first, second, third derivatives	5
5.2	PY2 - CFT Online B1+B2: Gaussian time shift	7
5.3	PY3 - July 2025 C1+C2: time compression plus frequency shift	7
5.4	PY4 - Online B1+B2: piecewise function and Parseval	8
5.5	PY5 - Online C1+C2: discover hidden sinusoids	8
5.6	PY6 - Online A1+A2: noisy image recovery	9
6	Fourteen new exam-style questions with full answers	9
6.1	N1 - Parameter-time shift vs spatial translation of an epicycle drawing	9
6.2	N2 - Rotate, scale, and translate the entire drawing	10
6.3	N3 - Time reversal of a complex drawing	10
6.4	N4 - Differentiate the epicycle drawing: velocity signal	10
6.5	N5 - Harmonic low-pass filtering of a drawing	11
6.6	N6 - Parseval for complex Fourier Series	11
6.7	N7 - 2D low-pass filtering: turn the edge detector into a blur	11
6.8	N8 - 2D spatial shift property	11
6.9	N9 - Conjugate symmetry of a real image	12
6.10	N10 - Notch filtering periodic vertical stripe noise	12
6.11	N11 - Direction-selective edge extraction	13
6.12	N12 - 2D cosine modulation creates two shifted spectrum copies	13
6.13	N13 - Anisotropic 2D scaling	13
6.14	N14 - 2D Parseval with nested trapezoids	13
7	Common exam pitfalls and how to diagnose them	14
7.1	1. Hz versus rad/s	14
7.2	2. Forward and inverse signs	14
7.3	3. Forgetting the Fourier-Series normalization	14
7.4	4. Forgetting negative Fourier-Series harmonics	14
7.5	5. Treating c_0 like an ordinary rotating epicycle	14
7.6	6. Wrong 2D array orientation	14

7.7	7. Wrong frequency axes	14
7.8	8. Integrating along the wrong axis	14
7.9	9. Sign mistakes after splitting real and imaginary parts	15
7.10	10. Raw phase MSE	15
7.11	11. Interpolating phase directly	15
7.12	12. Using index cutoff when the question gives physical frequency	15
7.13	13. Time shift implemented by <code>np.roll</code>	15
7.14	14. Image shift implemented by row/column roll	15
7.15	15. Cutoff interpretation in high-pass filtering	15
7.16	16. Recomputing the 2D CFT unnecessarily	15
7.17	17. Finite integration windows	15
7.18	18. Boundary cropping after scaling/shift	15
8	30–40 minute exam workflow	16
8.1	First 3 minutes: classify the property	16
8.2	Next 5 minutes: determine shapes and units	16
8.3	Main 20 minutes: implement the smallest correct path	16
8.4	Final 7–10 minutes: sanity checks	16
8.5	What to print if the question asks for numerical verification	16
9	Fast formula-to-code reference	17
10	Final self-test checklist	17

1 What the previous questions actually test

The supplied previous-year online questions repeatedly reuse the numerical Fourier machinery from the corresponding offline work and then add one theoretical property, one signal modification, or one filtering objective. The important pattern is therefore not “memorize an old question.” It is:

1. generate or modify a signal with the required class/framework;
2. compute the transform numerically with trapezoidal integration;
3. predict the result using a Fourier property;
4. compare numerical and theoretical results using plots and/or MSE;
5. explain the physical meaning of the spectral change.

Supplied question	Core task	What it trains
July 2025 A1+A2	First, second, third derivative	Property verification; magnitude/phase MSE
CFT Online B1+B2	Gaussian time shift	OOP extension; time-shift magnitude and phase
July 2025 C1+C2	Time compression + frequency shift	Combining two Fourier properties
Online B1+B2	Piecewise function + Parseval	FT/IFT reuse and energy equality
Online C1+C2	Hidden sinusoidal components	Spectral peak interpretation and reconstruction
Online A1+A2	Noisy image recovery	Frequency-domain filtering

This year’s offline assignment has two unusually reusable pieces of machinery: (1) complex Fourier Series coefficients for a 2D drawing and reconstruction by epicycles, and (2) separable forward/inverse 2D-CFT with frequency-domain filtering. Those are the most natural foundations for new online questions.

1.1 Likely question families for this year’s exam

The following is an inference from the supplied pattern, not a statement from the course staff:

1. Fourier-Series property verification: time shift, reversal, differentiation, Parseval, or coefficient manipulation.
2. Geometric transformations of the complex drawing: rotation, scale, spatial translation, change of starting point.
3. 2D filtering variants: low-pass blur, notch noise removal, band-pass or directional filtering.
4. 2D transform properties: spatial shift, modulation/frequency shift, scaling, conjugate symmetry, Parseval.
5. Numerical/debugging tasks involving `np.trapezoid`, array axes, phase wrapping, or reconstruction.

2 Essential theory you should be able to use immediately

2.1 Continuous Fourier Transform convention used in these questions

The supplied questions use frequency f in cycles per unit time:

$$X(f) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi ft} dt, \quad x(t) = \int_{-\infty}^{\infty} X(f)e^{j2\pi ft} df.$$

Do not mix this with the angular-frequency convention $X(j\omega)$, where the kernel is $e^{-j\omega t}$.

2.2 CFT property sheet

Assume $x(t) \leftrightarrow X(f)$.

$$\text{Time shift: } x(t - t_0) \leftrightarrow X(f)e^{-j2\pi ft_0}$$

$$\text{Frequency shift: } x(t)e^{j2\pi f_0 t} \leftrightarrow X(f - f_0)$$

$$\text{Time scaling: } x(at) \leftrightarrow \frac{1}{|a|} X\left(\frac{f}{a}\right)$$

$$\text{Combined: } x(at)e^{j2\pi f_0 t} \leftrightarrow \frac{1}{|a|} X\left(\frac{f - f_0}{a}\right)$$

$$\text{Differentiation: } \frac{d^m x}{dt^m} \leftrightarrow (j2\pi f)^m X(f)$$

$$\text{Time reversal: } x(-t) \leftrightarrow X(-f)$$

$$\text{Parseval: } \int |x(t)|^2 dt = \int |X(f)|^2 df.$$

Pitfall. For $x(t) = \cos(4t)$, the angular frequency is 4 rad/s, but the CFT peaks using the f convention are at $f = \pm 4/(2\pi) = \pm 2/\pi$ Hz.

2.3 Complex Fourier Series used by the epicycle assignment

For a periodic complex drawing $z(t) = x(t) + jy(t)$ of period T :

$$z(t) = \sum_{n=-\infty}^{\infty} c_n e^{jn\omega_0 t}, \quad \omega_0 = \frac{2\pi}{T},$$

$$c_n = \frac{1}{T} \int_0^T z(t) e^{-jn\omega_0 t} dt.$$

The finite reconstruction is

$$\hat{z}_N(t) = \sum_{n=-N}^N c_n e^{jn\omega_0 t}.$$

Coefficient properties that are especially exam-friendly:

$$z(t - t_0) : d_n = c_n e^{-jn\omega_0 t_0}$$

$$z(-t) : d_n = c_{-n}$$

$$\frac{dz}{dt} : d_n = jn\omega_0 c_n$$

$$az(t), a \in \mathbb{C} : d_n = ac_n$$

$$z(t) + d : d_0 = c_0 + d, \quad d_n = c_n \quad (n \neq 0)$$

$$Ae^{j\theta} z(t) + d : d_n = Ae^{j\theta} c_n \quad (n \neq 0), \quad d_0 = Ae^{j\theta} c_0 + d.$$

Parseval for the complex Fourier Series is

$$\frac{1}{T} \int_0^T |z(t)|^2 dt = \sum_{n=-\infty}^{\infty} |c_n|^2.$$

Pitfall. A complex drawing does not automatically satisfy $c_{-n} = c_n^*$. That conjugate symmetry is guaranteed for real-valued periodic signals, not arbitrary $x(t) + jy(t)$.

2.4 2D Continuous Fourier Transform

For an image-like continuous signal $I(x, y)$:

$$F(u, v) = \iint I(x, y) e^{-j2\pi(ux+vy)} dx dy.$$

The inverse is

$$I(x, y) = \iint F(u, v) e^{j2\pi(ux+vy)} du dv.$$

Useful 2D properties:

$$\begin{aligned} I(x - x_0, y - y_0) &\leftrightarrow F(u, v) e^{-j2\pi(ux_0+vy_0)} \\ I(x, y) e^{j2\pi(u_0x+v_0y)} &\leftrightarrow F(u - u_0, v - v_0) \\ I(ax, by) &\leftrightarrow \frac{1}{|ab|} F\left(\frac{u}{a}, \frac{v}{b}\right). \end{aligned}$$

For a real image,

$$F(-u, -v) = F^*(u, v).$$

2D Parseval is

$$\iint |I(x, y)|^2 dx dy = \iint |F(u, v)|^2 du dv.$$

2.5 Spatial frequency intuition

u measures variation as you move along x (across columns). v measures variation as you move along y (across rows). Therefore:

- large $|u|$: rapid left-right variation; useful for vertical boundaries/edges;
- large $|v|$: rapid up-down variation; useful for horizontal boundaries/edges;
- near $(u, v) = (0, 0)$: smooth/constant brightness;
- far from the origin: fine detail, sharp transitions, texture, and often noise.

3 Numerical Fourier coding patterns to memorize

3.1 1D CFT without FFT

```
def cft(t, x, f):
    kernel = np.exp(-1j * 2*np.pi * np.outer(f, t))
    return np.trapezoid(x[None, :] * kernel, t, axis=1)
```

For inverse CFT:

```
def icft(f, X, t):
    kernel = np.exp(1j * 2*np.pi * np.outer(t, f))
    return np.trapezoid(X[None, :] * kernel, f, axis=1)
```

3.2 Fourier Series coefficient and reconstruction

This is the central pattern in the supplied `fs_redrawer.py` solution:

```
def calculate_cn(self, n):
    kernel = np.exp(-1j*n*self.omega*self.t)
    return np.trapezoid(self.signal * kernel, self.t) / self.T
```

```
def calculate_all_coefficients(self):
    for n in range(-self.N, self.N + 1):
        self.coefs[n] = self.calculate_cn(n)

def approximate(self, t):
    result = 0
    for n, cn in self.coefs.items():
        result += cn * np.exp(1j*n*self.omega*t)
    return result
```

3.3 2D trapezoidal integration axis rule

If an image array has shape (N_y, N_x) , then rows correspond to y and columns to x . For a generic sampled function $G[y, x]$:

```
# integrate over x first -> one value per y
inner = np.trapezoid(G, x, axis=1)    # shape (Ny,)
# then integrate over y
value = np.trapezoid(inner, y)
```

Equivalently you may integrate y first, then x , if all axes and formulas are changed consistently.

3.4 Phase error: do not subtract raw angles blindly

Angles wrap at $\pm\pi$. For example, 179° and -179° differ by 2° , not 358° . Use

```
diff = measured_phase - predicted_phase
wrapped = np.angle(np.exp(1j * diff))
mask = predicted_magnitude > 1e-6 * predicted_magnitude.max()
mse_phase = np.mean(wrapped[mask]**2)
```

Pitfall. Phase is physically meaningless where the magnitude is essentially zero. Mask those bins before interpreting phase MSE. If an exam explicitly prints a raw-MSE formula, you can compute the requested number but still state that wrapped, magnitude-masked phase error is the numerically meaningful check.

3.5 Interpolation for transformed spectra

If a property requires $X((f - f_0)/a)$, the required frequency points usually do not equal your original grid. Best options:

1. compute the CFT directly on the transformed query frequencies; or
2. interpolate the real and imaginary parts separately, then rebuild the complex spectrum.

Do not interpolate wrapped phase directly unless you first unwrap it carefully.

4 This year's assignment: what every line is doing

4.1 Task 1: Fourier epicycles

The supplied assignment defines $z(t) = x(t) + jy(t)$ over one closed period and asks for all harmonics $-N \leq n \leq N$. The supplied solution correctly uses:

- $T = t[-1] - t[0]$;
- $\omega_0 = 2\pi/T$;
- numerical coefficient integration with `np.trapezoid`;

- a dictionary mapping $n \mapsto c_n$;
- reconstruction by directly summing $c_n e^{jn\omega_0 t}$.

Why negative harmonics matter: the drawing signal is complex, so positive and negative harmonics are independent degrees of freedom. Dropping $n < 0$ can heavily distort the path.

The coefficient c_0 is special:

$$c_0 = \frac{1}{T} \int_0^T z(t) dt,$$

so it is the average complex position, i.e. the center/offset of the traced path.

4.2 Task 2: separable 2D-CFT

The supplied solution first integrates each image row against cosine and sine kernels in x , producing intermediate real/imaginary arrays indexed by (y, u) . It then integrates those intermediates over y for every v .

Write the first stage as

$$A(y, u) = \int I(x, y) \cos(2\pi ux) dx, \quad B(y, u) = - \int I(x, y) \sin(2\pi ux) dx.$$

Then multiplying $A + jB$ by $\cos(2\pi vy) - j \sin(2\pi vy)$ gives

$$\Re F = \int [A \cos(2\pi vy) + B \sin(2\pi vy)] dy,$$

$$\Im F = \int [B \cos(2\pi vy) - A \sin(2\pi vy)] dy.$$

This exactly explains the signs in the supplied code.

4.3 Why `self.u` and `self.v` are not optional

The assignment constructs

$$u \in \left[-\frac{1}{2\Delta x}, \frac{1}{2\Delta x} \right], \quad v \in \left[-\frac{1}{2\Delta y}, \frac{1}{2\Delta y} \right].$$

Using x and y themselves as frequency axes would only cover a small, physically unrelated range and would miss edge-scale frequencies.

4.4 High-pass filter and inverse

The provided high-pass filter zeros a disk around the center of the sampled spectrum. It measures cutoff in *array-index radius*, not directly in physical (u, v) units. After low frequencies are removed, the inverse result has positive and negative values because average brightness/DC was removed. The given entry point therefore takes absolute value, normalizes, and inverts for display.

Pitfall. The names `cx` and `cy` in the provided filter are semantically swapped with row/column language, but the actual center indices are still used consistently in the distance expression. Focus on index geometry, not variable names.

5 Solved previous-year questions

5.1 PY1 - July 2025 A1+A2: first, second, third derivatives

Source basis: *Supplied file 220_online(1).pdf*.

Given

$$x(t) = 0.5 \cos(4t) + 0.5 \sin(6t),$$

and the CFT differentiation property

$$\mathcal{F} \left\{ \frac{dx}{dt} \right\} = j2\pi f X(f).$$

Step 1: derivatives in time.

$$\begin{aligned} x'(t) &= -2 \sin(4t) + 3 \cos(6t), \\ x''(t) &= -8 \cos(4t) - 18 \sin(6t), \\ x^{(3)}(t) &= 32 \sin(4t) - 108 \cos(6t). \end{aligned}$$

Therefore

$$X_1(f) = (j2\pi f)X(f), \quad X_2(f) = (j2\pi f)^2 X(f), \quad X_3(f) = (j2\pi f)^3 X(f).$$

Frequency locations. Because the sinusoids are written with angular frequencies 4 and 6 rad/s, the peaks are at

$$f = \pm \frac{2}{\pi}, \quad f = \pm \frac{3}{\pi}.$$

Analytical distribution form.

$$X(f) = \frac{1}{4} \left[\delta \left(f - \frac{2}{\pi} \right) + \delta \left(f + \frac{2}{\pi} \right) \right] + \frac{1}{4j} \left[\delta \left(f - \frac{3}{\pi} \right) - \delta \left(f + \frac{3}{\pi} \right) \right].$$

A finite numerical time window replaces ideal impulses by narrow lobes/peaks.

Minimal numerical solution.

```
t = np.linspace(-20, 20, 12000)
f = np.linspace(-2, 2, 4001)
x = 0.5*np.cos(4*t) + 0.5*np.sin(6*t)
x1 = -2*np.sin(4*t) + 3*np.cos(6*t)
x2 = -8*np.cos(4*t) - 18*np.sin(6*t)
x3 = 32*np.sin(4*t) - 108*np.cos(6*t)

X = cft(t, x, f)
Y1 = cft(t, x1, f)
Y2 = cft(t, x2, f)
Y3 = cft(t, x3, f)

P1 = (1j*2*np.pi*f) * X
P2 = (1j*2*np.pi*f)**2 * X
P3 = (1j*2*np.pi*f)**3 * X

for Y, P in [(Y1,P1), (Y2,P2), (Y3,P3)]:
    mse_mag = np.mean((np.abs(Y)-np.abs(P))**2)
    mask = np.abs(P) > 1e-5*np.abs(P).max()
    phase_err = np.angle(np.exp(1j*(np.angle(Y)-np.angle(P))))
    mse_phase = np.mean(phase_err[mask]**2)
    print(mse_mag, mse_phase)
```

Pitfall. Differentiation multiplies magnitude by $|2\pi f|^m$, so high-frequency numerical error is amplified. Use enough time/frequency samples.

5.2 PY2 - CFT Online B1+B2: Gaussian time shift

Source basis: Supplied file *CSE220_OnlineCFT_B1_B2(1).pdf*.

The question uses

$$x(t) = e^{-t^2}, \quad y(t) = x(t-1) = e^{-(t-1)^2}.$$

Theory:

$$Y(f) = X(f)e^{-j2\pi f}.$$

Hence

$$|Y(f)| = |X(f)|, \quad \angle Y(f) = \angle X(f) - 2\pi f \pmod{2\pi}.$$

The analytical Gaussian transform under this convention is

$$X(f) = \sqrt{\pi} e^{-\pi^2 f^2}.$$

Framework-style implementation.

```
def gaussian(self, a, shift=0.0):
    tau = self.t - shift
    return np.exp(-a * tau**2)

x = gen.gaussian(a=1, shift=0)
y = gen.gaussian(a=1, shift=1)
X = analyzer_x.compute_cft()
Y = analyzer_y.compute_cft()
pred = X * np.exp(-1j*2*np.pi*f*1.0)
```

Magnitude MSE should be very small. For phase, compare wrapped difference only where magnitude is significant.

Pitfall. A time shift is not `np.roll`. Rolling samples introduces circular/discrete-index behavior. The online explicitly asks you to use the OOP signal-generation framework.

5.3 PY3 - July 2025 C1+C2: time compression plus frequency shift

Source basis: Supplied file *July 2025 CSE 220 CFT Online(1).pdf*.

The verification formula printed in the question corresponds to

$$y(t) = x(at)e^{j2\pi f_0 t},$$

with $a = 10$ and $f_0 = 10$. Therefore

$$Y(f) = \frac{1}{|a|} X\left(\frac{f-f_0}{a}\right).$$

The magnitude and phase predictions are

$$|Y(f)| = \frac{1}{|a|} \left| X\left(\frac{f-f_0}{a}\right) \right|,$$

$$\angle Y(f) = \angle X\left(\frac{f-f_0}{a}\right) \quad (\text{where magnitude is nonzero}).$$

Clean numerical approach: directly evaluate X at the required frequencies.

```
f_query = (f - f0) / a
X_query = cft(t, x, f_query)
pred = X_query / abs(a)
Y = cft(t, y, f)
```

This avoids interpolation error.

Interpretation. Compressing time by factor 10 makes features ten times faster, so the spectrum spreads by factor 10 and scales down by $1/10$. Multiplication by $e^{j2\pi f_0 t}$ then shifts the whole spectrum by $+f_0$.

Pitfall. Order matters. If you frequency-shift first and then replace t by at , the effective modulation frequency also scales. Always derive the exact $y(t)$ you are implementing.

5.4 PY4 - Online B1+B2: piecewise function and Parseval

Source basis: Supplied file *Online_B1B2_CSE220.pdf*.

The PDF states that the outer intervals $[-3, -1]$ and $[1, 3]$ are parabolas. From the plotted graph, the piecewise function is consistent with

$$f(t) = \begin{cases} 0, & |t| > 3, \\ (t+3)^2, & -3 \leq t < -1, \\ 5 - |t|, & -1 \leq t \leq 1, \\ (3-t)^2, & 1 < t \leq 3. \end{cases}$$

This expression is inferred from the graph because the PDF does not print the explicit formula in text.

The time-domain energy can even be checked analytically:

$$E_t = 2 \int_0^2 u^4 du + 2 \int_0^1 (5-t)^2 dt = \frac{64}{5} + \frac{122}{3} = \frac{802}{15} \approx 53.4667.$$

Numerical solution.

```
def piecewise(t):
    x = np.zeros_like(t, dtype=float)
    m = (-3 <= t) & (t < -1)
    x[m] = (t[m] + 3)**2
    m = (-1 <= t) & (t <= 1)
    x[m] = 5 - np.abs(t[m])
    m = (1 < t) & (t <= 3)
    x[m] = (3 - t[m])**2
    return x

Et = np.trapezoid(np.abs(x)**2, t)
Ef = np.trapezoid(np.abs(X)**2, f)
print(Et, Ef, abs(Et-Ef))
```

Use a wide enough frequency range; otherwise E_f misses spectral tails and is too small.

5.5 PY5 - Online C1+C2: discover hidden sinusoids

Source basis: Supplied file *Online_C1C2_CSE220_FT.pdf*.

The given expression is

$$f(t) = 2 \sin(14\pi t) - \sin(2\pi t) (4 \sin(2\pi t) \sin(14\pi t) - 1).$$

Expand it:

$$\begin{aligned} f(t) &= 2 \sin(14\pi t) - 4 \sin^2(2\pi t) \sin(14\pi t) + \sin(2\pi t) \\ &= 2 \sin(14\pi t) - 2[1 - \cos(4\pi t)] \sin(14\pi t) + \sin(2\pi t) \\ &= 2 \cos(4\pi t) \sin(14\pi t) + \sin(2\pi t) \\ &= \sin(18\pi t) + \sin(10\pi t) + \sin(2\pi t). \end{aligned}$$

So the three sinusoidal frequencies are

$$1 \text{ Hz}, \quad 5 \text{ Hz}, \quad 9 \text{ Hz},$$

all with amplitude 1 and zero extra phase in sine form.

The numerical FT should show symmetric peaks at $\pm 1, \pm 5, \pm 9$ Hz. Reconstruct

$$g(t) = \sin(2\pi t) + \sin(10\pi t) + \sin(18\pi t)$$

and verify `np.mean((f-g)**2)` is near machine precision.

5.6 PY6 - Online A1+A2: noisy image recovery

Source basis: Supplied file `Online_CSE220_A1A2_FT.pdf`.

The PDF supplies the mission and a row-by-row FT hint, but the actual `noisy_image.png` is not among the uploaded materials. Therefore the exact secret letter cannot be determined from the PDF alone. The solvable part is the method.

If each row is a signal across horizontal position x :

```
# image shape: rows x cols
freq = np.linspace(-0.5, 0.5, image.shape[1])
Frows = np.zeros((image.shape[0], len(freq)), dtype=complex)
for r in range(image.shape[0]):
    Frows[r] = cft(x, image[r], freq)

# Average magnitude helps expose noise frequencies common to many rows.
score = np.mean(np.abs(Frows), axis=0)
# Ignore DC, locate strong symmetric off-center peaks, and zero a small band.

for r in range(image.shape[0]):
    recovered[r] = np.real(icft(freq, Frows_filtered[r], x))
```

Periodic vertical stripe noise varies rapidly across x and therefore produces strong off-center horizontal-frequency peaks. Notch those frequencies and inverse-transform row by row.

Pitfall. Do not automatically remove the globally strongest peak: DC/low frequency is often strongest and contains the image's average brightness. Search for unusually strong symmetric off-center peaks.

6 Fourteen new exam-style questions with full answers

These are not copies of the supplied previous-year questions. They are deliberately constructed from the current assignment's machinery and the transformation/filtering style used in prior online exams.

6.1 N1 - Parameter-time shift vs spatial translation of an epicycle drawing

Question. Let $z(t)$ be a periodic complex drawing with coefficients c_n . Compare:

$$z_1(t) = z(t - t_0), \quad z_2(t) = z(t) + d.$$

Predict the coefficients and explain the geometric difference.

Answer. For the parameter shift,

$$d_n = c_n e^{-jn\omega_0 t_0}.$$

Every harmonic keeps the same magnitude but changes phase. Geometrically, the same closed curve is traced with a different starting point/parameter origin.

For spatial translation,

$$e_0 = c_0 + d, \quad e_n = c_n \quad (n \neq 0).$$

The entire curve moves by the same complex displacement d ; only the DC coefficient changes.

Verification core.

```
pred_shift = {n: c[n]*np.exp(-1j*n*w0*t0) for n in c}
pred_trans = c.copy()
pred_trans[0] = c[0] + d
```

Pitfall. These two operations are easy to confuse because both are called a “shift.” Time/parameter shift changes harmonic phases. Spatial translation changes only c_0 .

6.2 N2 - Rotate, scale, and translate the entire drawing

Question. Construct

$$y(t) = Ae^{j\theta}z(t) + d.$$

Find the Fourier coefficients of y without reintegrating from scratch, then verify numerically.

Answer. By linearity,

$$y_n = Ae^{j\theta}c_n \quad (n \neq 0),$$

$$y_0 = Ae^{j\theta}c_0 + d.$$

Multiplying by A changes every epicycle radius by A . Multiplying by $e^{j\theta}$ rotates every complex vector by θ . Adding d changes only the center.

```
a = A*np.exp(1j*theta)
pred = {n: a*cn for n, cn in c.items()}
pred[0] += d
```

6.3 N3 - Time reversal of a complex drawing

Question. Let $y(t) = z(-t)$. Show the coefficient relationship and state what happens to tracing direction.

Answer.

$$d_n = c_{-n}.$$

The same geometric path is traced in the opposite parameter direction. Positive- and negative-index coefficients swap roles.

Derivation:

$$d_n = \frac{1}{T} \int_0^T z(-t)e^{-jn\omega_0 t} dt.$$

With a periodic change of variable, this becomes the coefficient at index $-n$.

Pitfall. Do not replace c_{-n} by c_n^* unless the original signal is real-valued. The SVG drawing signal is generally complex-valued.

6.4 N4 - Differentiate the epicycle drawing: velocity signal

Question. Define $v(t) = dz/dt$. Predict the coefficient v_n and explain the visual/spectral effect.

Answer.

$$v_n = jn\omega_0 c_n.$$

The DC term disappears because $n = 0$. The magnitude of harmonic n is multiplied by $|n|\omega_0$, so high-order harmonics become much more prominent. That is why differentiation emphasizes sharp, rapid details.

```
v_coeffs = {n: 1j*n*w0*cn for n, cn in coeffs.items()}
v_hat = sum(v_coeffs[n]*np.exp(1j*n*w0*t) for n in v_coeffs)
```

6.5 N5 - Harmonic low-pass filtering of a drawing

Question. After computing coefficients up to $N = 20$, reconstruct using only $|n| \leq 3$, then $|n| \leq 8$, then all harmonics. What changes?

Answer. A harmonic low-pass is simply

```
def keep_low(coeffs, K):
    return {n: (cn if abs(n) <= K else 0) for n, cn in coeffs.items()}
```

Small K keeps the broad, smooth shape but removes small-scale corners/details. Increasing K lowers reconstruction MSE. The coefficient c_0 controls the spatial average/center, so removing it would recenter the path near the origin rather than merely smoothing it.

6.6 N6 - Parseval for complex Fourier Series

Question. Verify

$$\frac{1}{T} \int_0^T |z(t)|^2 dt = \sum_n |c_n|^2$$

for increasing harmonic limit N .

Answer.

```
E_time = np.trapezoid(np.abs(z)**2, t) / T
E_coeff = sum(abs(cn)**2 for cn in coeffs.values())
rel_err = abs(E_time-E_coeff) / E_time
```

If the test signal contains only harmonics up to $|n| = 7$, then once $N \geq 7$ the numerical coefficient energy should nearly equal time-domain average power. For smaller N , the coefficient sum is the power captured by the retained harmonics.

Pitfall. For Fourier Series, this form is average power over one period, not total energy over all time. A nonzero periodic signal has infinite total energy if extended forever.

6.7 N7 - 2D low-pass filtering: turn the edge detector into a blur

Question. Replace the assignment's high-pass filter with a low-pass disk and reconstruct for several cutoff radii.

Answer.

```
def low_pass(real, imag, cutoff):
    rows, cols = real.shape
    cr, cc = rows//2, cols//2
    rr, cc_grid = np.ogrid[:rows, :cols]
    mask = (rr-cr)**2 + (cc_grid-cc)**2 <= cutoff**2
    return np.where(mask, real, 0), np.where(mask, imag, 0)
```

A small cutoff keeps only smooth intensity variation, producing strong blur. A larger cutoff restores more edges and texture.

Pitfall. The provided assignment filter cutoff is in array-index distance from the spectrum center. If an exam instead gives a cutoff in physical frequency units, build the mask from U, V values, not row/column indices.

6.8 N8 - 2D spatial shift property

Question. For

$$J(x, y) = I(x - x_0, y - y_0),$$

verify the 2D shift theorem.

Answer.

$$G(u, v) = F(u, v)e^{-j2\pi(ux_0+vy_0)}.$$

Therefore magnitude is unchanged and phase gains a plane:

$$\angle G = \angle F - 2\pi(ux_0 + vy_0).$$

```
U, V = np.meshgrid(u, v)
pred_real = real*np.cos(2*np.pi*(U*x0+V*y0)) \
            + imag*np.sin(2*np.pi*(U*x0+V*y0))
pred_imag = imag*np.cos(2*np.pi*(U*x0+V*y0)) \
            - real*np.sin(2*np.pi*(U*x0+V*y0))
```

The signs above come from multiplying $(R + jI)e^{-j\phi}$.

Pitfall. If you generate the shifted image by array rolling, you are testing discrete circular shift, not the continuous spatial shift specified by the theorem. For a clean numerical test, use an analytic image such as a Gaussian and evaluate it at $(x - x_0, y - y_0)$.

6.9 N9 - Conjugate symmetry of a real image

Question. Verify the property of the 2D CFT of a real image.

Answer.

$$F(-u, -v) = F^*(u, v).$$

With symmetric frequency arrays generated by `linspace`, reversing both indices corresponds to negating both frequency coordinates:

```
real_flip = real[::-1, ::-1]
imag_flip = imag[::-1, ::-1]
mse_real = np.mean((real - real_flip)**2)
mse_imag = np.mean((imag + imag_flip)**2)
```

Real part is even under simultaneous (u, v) reversal; imaginary part is odd.

6.10 N10 - Notch filtering periodic vertical stripe noise

Question. Add

$$n(x, y) = 0.25 \cos(2\pi u_0 x)$$

to a clean image. Remove it in frequency domain.

Answer. The cosine contributes two symmetric components at $(u, v) = (\pm u_0, 0)$. Use two small notch disks, not a central high-pass disk.

```
U, V = np.meshgrid(u, v)
r = 0.8
notch = (((U-u0)**2 + V**2) < r**2) | \
        (((U+u0)**2 + V**2) < r**2)
real_f = real.copy(); imag_f = imag.copy()
real_f[notch] = 0
imag_f[notch] = 0
```

After inverse CFT, compare MSE against the known clean synthetic image. The filtered MSE should be much lower.

Why vertical stripes give u -axis peaks. Their intensity changes while moving left-right (x), but is nearly constant while moving up-down (y). Hence the noise has nonzero u frequency and approximately $v = 0$.

6.11 N11 - Direction-selective edge extraction

Question. Design one mask for vertical edges and one for horizontal edges.

Answer. Vertical boundaries change rapidly as x changes, so emphasize large $|u|$. Horizontal boundaries change rapidly as y changes, so emphasize large $|v|$.

```
U, V = np.meshgrid(u, v)
vertical_mask = np.abs(U) >= u_cut
horizontal_mask = np.abs(V) >= v_cut
```

For stronger directional selectivity, also restrict the perpendicular direction, e.g.

```
vertical_mask = (np.abs(U) >= u_cut) & (np.abs(V) <= v_band)
```

This is still frequency-domain edge extraction; no Sobel/Prewitt/Canny is needed.

6.12 N12 - 2D cosine modulation creates two shifted spectrum copies

Question. Let

$$J(x, y) = I(x, y) \cos(2\pi(u_0x + v_0y)).$$

Predict its spectrum.

Answer. Since

$$\cos(2\pi(u_0x + v_0y)) = \frac{1}{2}e^{j2\pi(u_0x + v_0y)} + \frac{1}{2}e^{-j2\pi(u_0x + v_0y)},$$

we get

$$G(u, v) = \frac{1}{2}F(u - u_0, v - v_0) + \frac{1}{2}F(u + u_0, v + v_0).$$

A spectrum originally centered at the origin appears as two half-amplitude copies centered at $\pm(u_0, v_0)$.

Pitfall. A real cosine produces two copies. A complex exponential produces one shifted copy.

6.13 N13 - Anisotropic 2D scaling

Question. Let

$$J(x, y) = I(ax, by).$$

Predict the spectrum and explain the width changes for $a = 2$, $b = 0.5$.

Answer.

$$G(u, v) = \frac{1}{|ab|} F\left(\frac{u}{a}, \frac{v}{b}\right).$$

For $a = 2$, the image is compressed along x , so its spectrum spreads along u . For $b = 0.5$, the image expands along y , so its spectrum narrows along v . Here $|ab| = 1$, so there is no overall amplitude factor.

Pitfall. When verifying numerically on finite grids, compare only where $(u/a, v/b)$ lies inside the sampled F grid. Otherwise interpolation or extrapolation error dominates.

6.14 N14 - 2D Parseval with nested trapezoids

Question. Verify

$$\iint |I(x, y)|^2 dx dy = \iint |F(u, v)|^2 du dv.$$

Answer. If `I.shape == (len(y), len(x))` and `real.shape == (len(v), len(u))`:

```

E_xy = np.trapezoid(
    np.trapezoid(I**2, x, axis=1),
    y
)

mag2 = real**2 + imag**2
E_uv = np.trapezoid(
    np.trapezoid(mag2, u, axis=1),
    v
)

```

The equality is approximate because both spatial and frequency integrals are truncated and sampled. A larger frequency window/density generally improves the frequency-domain energy estimate until numerical cost/roundoff become limiting.

7 Common exam pitfalls and how to diagnose them

7.1 1. Hz versus rad/s

If the transform kernel contains $2\pi f$, then f is cycles/unit. A term $\cos(\omega_0 t)$ appears at $f_0 = \omega_0/(2\pi)$.

7.2 2. Forward and inverse signs

Forward CFT/FS coefficient kernel: negative sign. Reconstruction/inverse kernel: positive sign.

$$e^{-j\cdot} \text{ for analysis, } \quad e^{+j\cdot} \text{ for synthesis.}$$

7.3 3. Forgetting the Fourier-Series normalization

Complex Fourier Series coefficients need $1/T$. The CFT does not.

7.4 4. Forgetting negative Fourier-Series harmonics

The current assignment explicitly needs $n = -N, \dots, N$. For a complex drawing, negative harmonics are not redundant.

7.5 5. Treating c_0 like an ordinary rotating epicycle

$n = 0$ does not rotate. It is a constant complex offset: the average location of the path.

7.6 6. Wrong 2D array orientation

For an image, `I[y, x]` means row y , column x . Thus `shape[0]` corresponds to y and `shape[1]` to x .

7.7 7. Wrong frequency axes

Use `self.u` and `self.v` for 2D CFT. The assignment explicitly constructs them from $\Delta x, \Delta y$ to cover the Nyquist range.

7.8 8. Integrating along the wrong axis

If you multiply each row by an x -kernel, integrate with `axis=1`. If you are integrating a (v, u) spectrum over u , again the inner integration is typically `axis=1`.

7.9 9. Sign mistakes after splitting real and imaginary parts

Memorize complex multiplication:

$$(a + jb)(c - jd) = (ac + bd) + j(bc - ad),$$

$$(a + jb)(c + jd) = (ac - bd) + j(ad + bc).$$

These two identities generate almost every sign in forward/inverse 2D-CFT.

7.10 10. Raw phase MSE

Phase wraps at $\pm\pi$ and is meaningless near zero magnitude. Wrap the difference and mask weak bins.

7.11 11. Interpolating phase directly

Interpolate complex real/imaginary components or compute the transform directly at target frequencies. Wrapped phase has discontinuities.

7.12 12. Using index cutoff when the question gives physical frequency

The provided `FrequencyFilter.high_pass` uses pixel-index radius. A question mentioning $u = 10$ or $v = 5$ requires a mask based on actual `self.u/self.v` values.

7.13 13. Time shift implemented by `np.roll`

`np.roll` is a circular discrete sample shift. It is not the same as evaluating a continuous formula at $t - t_0$.

7.14 14. Image shift implemented by row/column roll

Same issue in 2D. Use an analytic shifted test signal or interpolation if the theorem is continuous.

7.15 15. Cutoff interpretation in high-pass filtering

A higher high-pass cutoff removes a larger central disk. That removes more low/mid frequency content and usually leaves thinner/noisier high-frequency structure. For a low-pass filter, a higher cutoff keeps more detail.

7.16 16. Recomputing the 2D CFT unnecessarily

The supplied `plot_magnitude` calls `compute_cft()` again. In an exam with strict time, cache/reuse results if the skeleton permits it. If you are not allowed to add attributes, pass or reuse local arrays instead of modifying class structure.

7.17 17. Finite integration windows

CFT theory integrates over infinite time/space and frequency. Numerical code integrates finite windows. Therefore ideal zeros, perfect impulses, and exact Parseval equality become approximations.

7.18 18. Boundary cropping after scaling/shift

If a shifted or expanded Gaussian is partly outside $[-1, 1]$, the finite spatial window changes the signal itself. Use a small shift or sufficiently localized analytic signal when verifying theory.

8 30–40 minute exam workflow

8.1 First 3 minutes: classify the property

Write the theoretical relationship before coding. For example:

- derivative $\Rightarrow (j2\pi f)^m$;
- time shift $\Rightarrow e^{-j2\pi ft_0}$;
- FS parameter shift $\Rightarrow e^{-jn\omega_0 t_0}$;
- spatial translation of drawing $\Rightarrow c_0$ only;
- image shift $\Rightarrow e^{-j2\pi(ux_0+vy_0)}$;
- high-pass/low-pass/notch $\Rightarrow$ create a mask and inverse-transform.

8.2 Next 5 minutes: determine shapes and units

Write comments like:

```
# image I: (Ny, Nx)
# x: (Nx,), y: (Ny,)
# real/imag spectrum: (Nv, Nu)
# u: (Nu,), v: (Nv,)
```

This prevents many 2D mistakes.

8.3 Main 20 minutes: implement the smallest correct path

Reuse the offline methods. Do not rewrite a transform if the question only asks for a property or filter. Keep the new code isolated: one signal-generation method, one property prediction, one mask, or one MSE block.

8.4 Final 7–10 minutes: sanity checks

Use tests that expose sign/unit errors quickly:

- time shift: magnitudes should overlap;
- derivative: DC should become zero;
- low-pass: image should get smoother;
- high-pass: smooth background should disappear;
- notch: specific symmetric spectral peaks should vanish;
- real image: conjugate symmetry should approximately hold;
- FS translation: only c_0 should change;
- inverse transform: output shape must equal original shape.

8.5 What to print if the question asks for numerical verification

At minimum:

```
print("Magnitude MSE:", mse_mag)
print("Phase MSE:", mse_phase)
print("Relative error:", rel_err)
```

If the result is not near zero, do not simply state “verified.” State the likely reason: finite window, interpolation, insufficient samples, phase wrap, or an implementation error.

9 Fast formula-to-code reference

Operation	Theory	Code thought
CFT time shift	$X(f)e^{-j2\pi ft_0}$	multiply complex spectrum by phase factor
CFT derivative	$(j2\pi f)^m X(f)$	complex pointwise multiplication
CFT scaling	$ a ^{-1}X(f/a)$	transformed frequency query/interpolation
CFT modulation	$X(f - f_0)$	query shifted frequency
FS time shift	$c_n e^{-jn\omega_0 t_0}$	loop/dict over n
FS reversal	c_{-n}	swap dictionary indices
FS derivative	$j n \omega_0 c_n$	multiply each coefficient
FS spatial translation	$c_0 \leftarrow c_0 + d$	change one dictionary entry
2D image shift	phase plane	use <code>meshgrid(u,v)</code>
2D low/high-pass	central mask	index-radius or physical U, V mask
2D notch	small masks at peaks	zero symmetric off-center regions
2D directional filter	select $ u $ or $ v $	vertical edges $\leftrightarrow$ high $ u $
2D Parseval	equal double integrals	nested <code>np.trapezoid</code>

10 Final self-test checklist

Before the exam, you should be able to do each item without notes:

1. Write CFT and inverse CFT with the correct 2π and signs.
2. Convert between angular frequency ω and frequency f .
3. Write FS c_n and reconstruct using $-N \dots N$.
4. Explain what c_0 means geometrically.
5. Derive time-shift, derivative, scaling, and modulation predictions.
6. Explain parameter shift versus spatial translation for a complex drawing.
7. Use wrapped phase error and a magnitude mask.
8. Use `np.trapezoid` on 2D arrays with the correct axis.
9. Explain why high-pass filtering finds edges and low-pass filtering blurs.
10. Explain why vertical stripes/edges relate to u and horizontal stripes/edges relate to v .
11. Build a radial, notch, and directional frequency mask.
12. State 2D conjugate symmetry for a real image.
13. Write 1D and 2D Parseval checks.
14. Identify when numerical discrepancies come from finite windows rather than wrong theory.

Recommended practice order: N1 $\rightarrow$ N4 $\rightarrow$ N6 $\rightarrow$ N8 $\rightarrow$ N10 $\rightarrow$ N11 $\rightarrow$ N14, then complete the remaining questions under a 30–40 minute timer.